

APUNTS COMPLETS COLLECTIONS + INTERFÍCIES (JAVA) – DAW1

1. CONCEPTE CLAU: INTERFÍCIES vs CLASSES

◆ Interfície

- ✗ No es pot instanciar
- ✓ Defineix **què es pot fer**
- ✓ Té mètodes sense cos

◆ Classe

- ✓ Implementa la interfície
 - ✓ Defineix **com es fa**
-

EXEMPLE DE LA “CAIXA”

Collection = una caixa

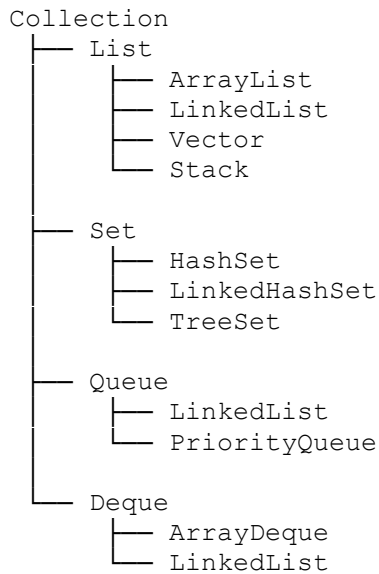
- La interfície diu:
 - “pots afegir coses”
 - “pots eliminar”
 - La classe diu:
 - “com ho faig internament”
-

Una **Collection** és una estructura que permet **guardar múltiples objectes**.

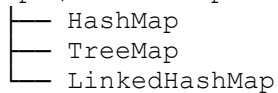
Idea clau:

- No treballes amb arrays → treballes amb **estructures dinàmiques**
- No accedeixes directament → utilitzes **mètodes**

2. 🌳 ESQUEMA GENERAL COLLECTIONS



Map (NO forma part de Collection)



📌 RESUM DIFERÈNCIES

Estructura	Duplicats	Ordenació	Accés
List	✓	✓	index
Set	✗	depèn	no index
Map	✗ clau única	depèn	clau
Queue	✓	FIFO	no index
Stack	✓	LIFO	no index

📌 MAP (clau → valor) ! IMPORTANT

- HashMap
 - TreeMap
- 👉 NO hereta de Collection

3. ◆ DECLARACIÓ GENÈRICA (IMPORTANT)

```
List<String> noms = new LinkedList<>();
```

👉 Per què?

- ✓ Programem contra la interfície
- ✓ Més flexible

! CAST?

👉 NO cal normalment

Només si vols mètodes específics:

```
LinkedList<String> l = (LinkedList<String>) noms;
```

4. ◆ OPERACIONS BÀSIQUES COLLECTIONS

```
add()  
remove()  
contains()  
size()  
isEmpty()  
clear()  
iterator()
```

◆ Altres importants

```
Collections.sort(llista);           // ordenar  
Collections.reverse(llista);        // invertir  
Collections.shuffle(llista);         // barrejar  
Collections.swap(llista, i, j);      // intercanviar  
Collections.frequency(l, x);         // comptar aparicions
```

Exemple: DAUS

```
Collections.frequency(llistaTirades, i);
```

👉 compta quantes vegades surt un valor

5. ♦ RECORREGUTS (MOLT IMPORTANT 🔥)

1 FOR clàssic

```
for(int i=0; i<llista.size(); i++){  
    System.out.println(llista.get(i));  
}
```

✓ Accés per índex

✗ Només per List

2 FOR LOOP

```
for(String s : llista){  
    System.out.println(s);  
}
```

✓ Més net

✗ Sense índex

3 forEach (LAMBDA)

```
llista.forEach(System.out::println);
```

👉 Equivalent:

```
llista.forEach(x -> System.out.println(x));
```

📌 :: = referència a mètode

4 ITERATOR

```
Iterator<String> it = llista.iterator();
```

```
while(it.hasNext()){  
    System.out.println(it.next());  
}
```

👉 Objectes per recórrer col·leccions

📌 IMPORTANT:

- ! És d'un sol ús
- ! No es pot reutilitzar

COMPARACIÓ

Mètode	Índex	Facilitat	Ús
for	✓	mitjà	List
foreach	✗	fàcil	general
forEach	✗	molt fàcil	modern
iterator	✗	avançat	eliminar segur

6. LAMBDA (IMPORTANT)

 Forma curta de funcions

```
(o1, o2) -> o1.getX() - o2.getX()
```

Exemple ordenar

```
llista.sort((a, b) -> a.getEdat() - b.getEdat());
```

 Evita fer classe Comparator

7. STREAMS (RESUM BÀSIC)

 Permeten treballar amb col·leccions de forma funcional

Exemple

```
llista.stream()
    .filter(x -> x.getPreu() > 10)
    .forEach(System.out::println);
```

Idees clau

- `stream()` → iniciar
- `filter()` → filtrar
- `forEach()` → executar

8. LIST (ArrayList / LinkedList)

 Permet:

- Duplicats ✓
- Accés per posició ✓

```
List<String> fruites = new ArrayList<>();
```

Operacions

```
fruites.add("poma");           // afegir
fruites.get(0);                // obtenir
fruites.set(1, "kiwi");        // modificar
fruites.remove(2);             // eliminar per index
fruites.remove("poma");        // eliminar per valor
fruites.size();                // mida
fruites.contains("kiwi");      // existeix?
```

9. MAP

IDEA

 clau → valor

! No pot haver-hi claus duplicades

```
Map<Integer, Integer> freq = new HashMap<>();
```

Operacions bàsiques

```
map.put(clau, valor);
map.get(clau);
map.containsKey(clau);
map.containsValue(valor);
```

Patró IMPORTANT

```
if (map.containsKey(k)) {
    map.put(k, map.get(k) + 1);
} else {
    map.put(k, 1);
}
```

◆ Recorregut

```
for (Map.Entry<K,V> e : map.entrySet()) {  
    e.getKey();        //Retorna la clau  
    e.getValue();      //Retorna el valor  
}
```

◆ Altres recorreguts

```
map.keySet()           //Retorna totes les claus del map  
map.values()           //Retorna tots els valors del map
```

10. ◆ SET

```
Set<Llapis> s = new HashSet<>();
```

👉 elimina duplicats AUTOMÀTICAMENT

11. ◆ QUEUE (CUES)

👉 FIFO → primer en entrar, primer en sortir

```
Queue<String> q = new LinkedList<>();
```

◆ Operacions

```
add()      // afegir  
peek()     // mirar primer  
remove()   // treure
```

12. ◆ STACK (PILA)

👉 LIFO → últim en entrar, primer en sortir

```
Stack<String> pila = new Stack<>();
```

◆ Operacions

```
push()      //Afegeix un element al TOP de la pila  
pop()       //Elimina i retorna l'element del TOP  
peek()      //Mostra l'element del TOP
```

13. ♦ DEQUE

👉 doble cua

- afegir davant/darrere
- treure davant/darrere

```
Deque<String> d = new ArrayDeque<>();
```

14. ♦ PRIORITY QUEUE

👉 ordena automàticament

```
PriorityQueue<Compra> pq = new PriorityQueue<>();
```

👉 usa Comparable

15. ♦ equals() i contains() 🔥

👉 Per defecte:

- compara adreça de memòria
-

! IMPORTANT

👉 contains() i remove() fan servir equals()

♦ IMPLEMENTACIÓ

```
@Override
public boolean equals(Object o){
    if(this == o) return true;
    if(!(o instanceof Producte)) return false;

    Producte altre = (Producte) o;
    return this.codiBarres.equalsIgnoreCase(altre.codiBarres);
}
```


16. hashCode()

👉 Necessari per:

- HashMap
 - HashSet
-

◆ IDEA

- Iguals → mateix hash
 - Diferents → hash diferent
-

◆ Exemple

```
@Override
public int hashCode() {
    return codiBarres.hashCode(); //mateix hashCode que codiBarres
}
```

equals vs hashCode

equals	hashCode
compara objectes	dona número
diu si són iguals	optimitza

17. COMPARABLE vs COMPARATOR

◆ Comparable (ordenació natural)

👉 dins la classe

```
public class Llapis implements Comparable<Llapis>{
    public int compareTo(Llapis o){
        return this.color - o.getColor();
    }
}
```

👉 ús:

```
Collections.sort(llista);
llista.sort(null);
```

◆ Comparator (ordenació externa)

👉 fora de la classe

```
import java.util.Comparator;

public class PerColor implements Comparator<Llapis>{
    public int compare(Llapis o1, Llapis o2){
        return o1.getColor() - o2.getColor();
    }
}
```

👉 ús:

```
Collections.sort(llista, new PerColor());
```

◆ També es pot fer dins del codi (classe anònima)

✓ NO cal crear classe externa

```
Collections.sort(llista, new Comparator<Llapis>() {
    public int compare(Llapis o1, Llapis o2){
        return o1.getGruix() - o2.getGruix();
    }
});
```

◆ Amb lambda

```
llista.sort((a, b) -> a.getColor() - b.getColor());
```

VS DIFERÈNCIES

Comparable	Comparator
dins classe	fora
1 criteri	molts criteris
modifica classe	NO modifica
compareTo	compare

📌 IMPORTANT:

- Comparacions simples → Comparable
 - Complexes → Comparator
-

18. PATRONS CLAU EXAMEN

Afegir o incrementar en Map

```
if (map.containsKey(k)) { //Si existeix
    map.put(k, map.get(k)+1); //Incrementa
} else {
    map.put(k, 1); //Afageix
}
```

Comptador Map

```
map.put(k, map.getOrDefault(k, 0)+1); // agafa valor o 0 si no existeix
```

Recórrer Map

```
for (Map.Entry<K,V> e : map.entrySet()) {
    e.getKey();
    e.getValue();
}
```

Ordenar llista

```
Collections.sort(llista);
```

Comparator ràpid (Lambda)

```
llista.sort((a,b) -> a.getX() - b.getX());
```

19. ERRORS TÍPICS

- ✗ Oblidar equals() i hashCode()
- ✗ Confondre remove(index) i remove(object)
- ✗ Pensar que Map permet claus duplicades
- ✗ No inicialitzar una llista
- ✗ Fer Collections.copy() amb llista petita
- ✗ Iterator reutilitzat

20. ♦ CAS PRÀCTICA SUPERMERCAT (MOLT IMPORTANT)

👉 Has treballat:

✓ Map → carro compra

- clau → producte
- valor → quantitat

✓ containsKey()

- per saber si el producte ja existeix

✓ Comparable

- ordenar productes (ex: caducitat)

✓ Comparator

- ordenar tèxtils per composició

✓ Set

- evitar duplicats

✓ Streams + lambda

- cercar producte per codi

21. TIPS EXAMEN

PART ESCRITA

👉 Estratègia:

- Escriu **clar i estructurat**
- Usa paraules clau:
 - “interficie”
 - “implementa”
 - “clau-valor”
 - “ordenació natural”

👉 Exemple bona resposta:

“Un Map associa una clau única amb un valor i no permet claus duplicades”

PART ORDINADOR

👉 ORDEN PRIORITARI:

1. Crear estructures correctes
2. Fer que funcioni
3. Després millorar

! NO siguis perfeccionista al principi

GESTIÓ TEMPS (IMPORTANT PER TU)

👉 Regla d'or:

🕒 70% temps → que funcioni

🕒 30% temps → millorar

PERFECCIONISME

👉 Regla:

- Primer funcional
- Després bonic

 TRUC PROFESSIONAL

Si et bloqueges:

👉 Escribe això:

```
// TODO
```

i continua

 RESUM FINAL

Has de dominar:

- ✓ List
- ✓ Map 🔥
- ✓ Set
- ✓ Comparable / Comparator 🔥
- ✓ equals + hashCode 🔥
- ✓ Collections.sort, frequency
- ✓ containsKey
- ✓ recorreguts
- ✓ lambda